Modules

Modules

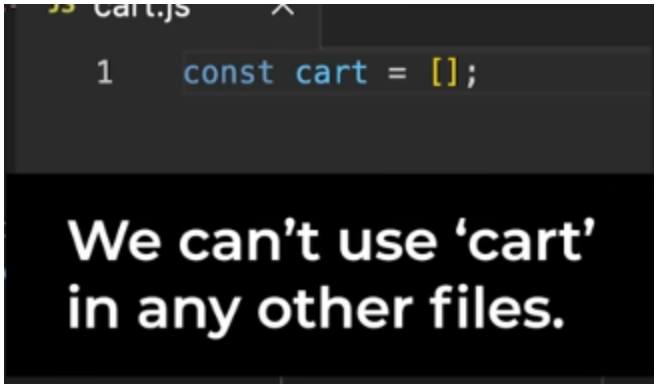
= a better way to organize our code

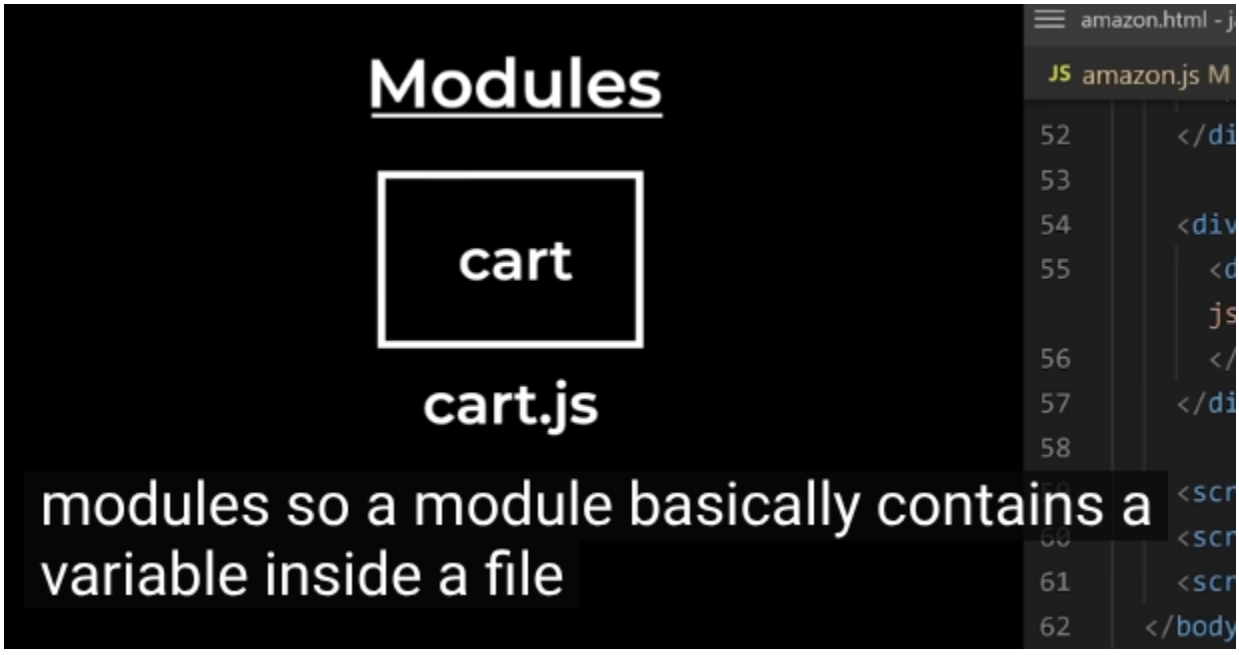
```
57      </div>
58
59      <script src="data/cart.js"></script>
60      <script src="data/products.js"></script>
61      <script src="scripts/amazon.js"></script>
62    </body>
63  </html>
```

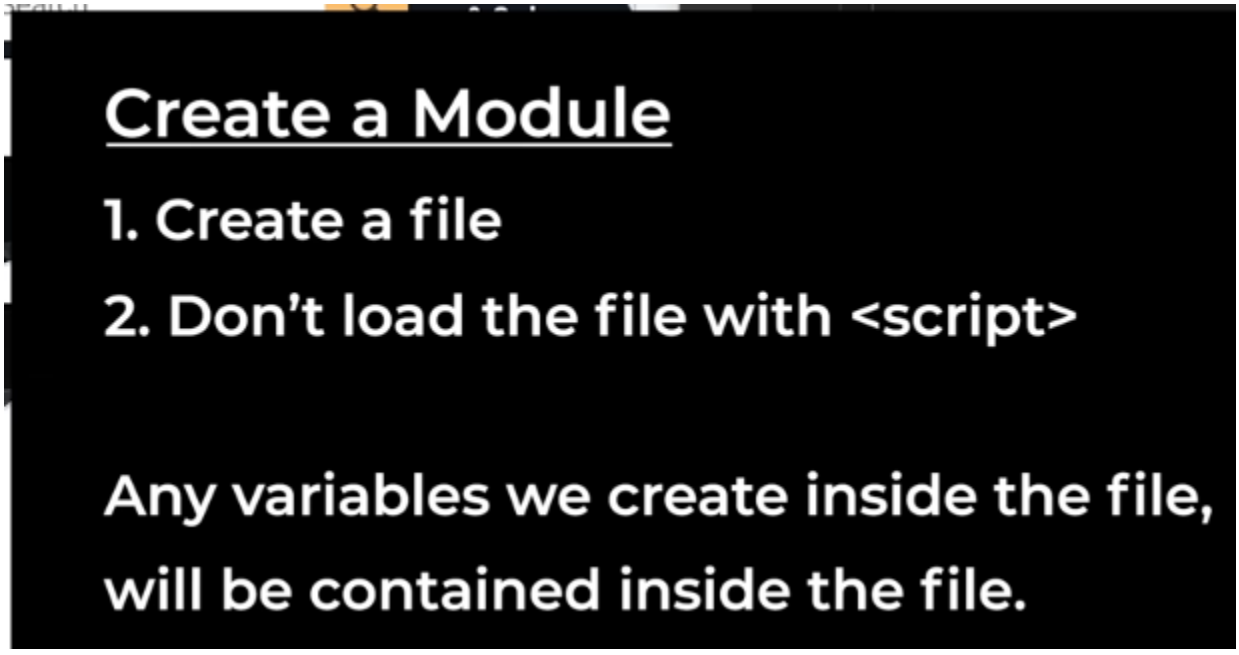
1. Combine all the files together into 1 big file.
2. Run all the code.

```
/body>
tml>
```

This can cause naming conflicts


cart.js
1 const cart = [];
We can't use 'cart' in any other files.


Modules
cart
cart.js
modules so a module basically contains a variable inside a file
amazon.html - j
JS amazon.js M
52 </di
53
54 <div
55 <d
js
56 </
57 </di
58
61 <scr
62 </body


Create a Module
1. Create a file
2. Don't load the file with <script>
Any variables we create inside the file, will be contained inside the file.

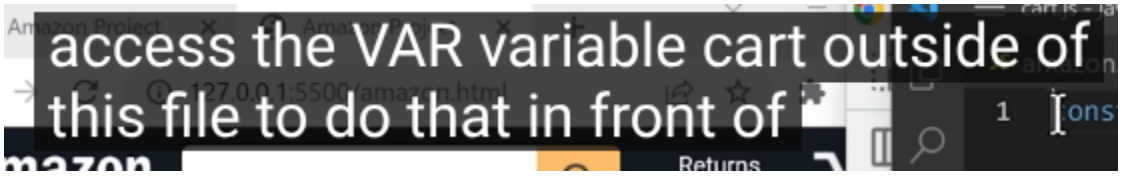
Get a Variable Out of a File

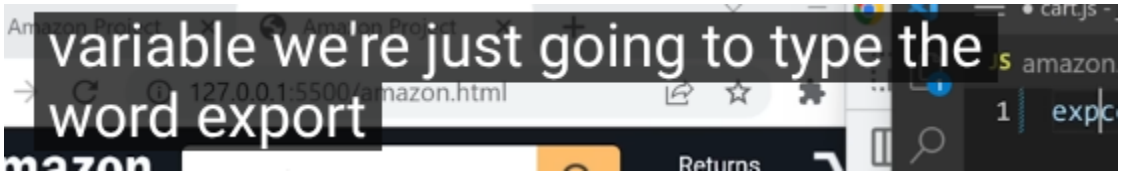
1. Add type="module" attribute
2. Export
3. Import

```
</div>
<script src="data/products.js"></script>
<script type="module" src="scripts/amazon.
js"></script>
```

type="module" attribute

Let's this file get variables out of other files.

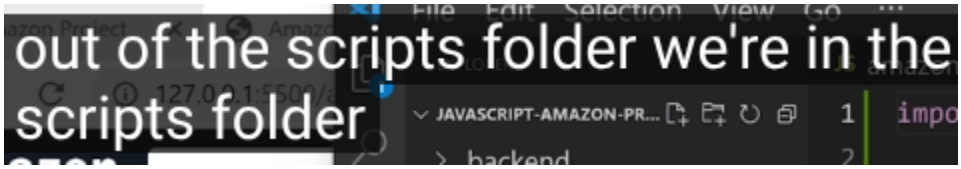


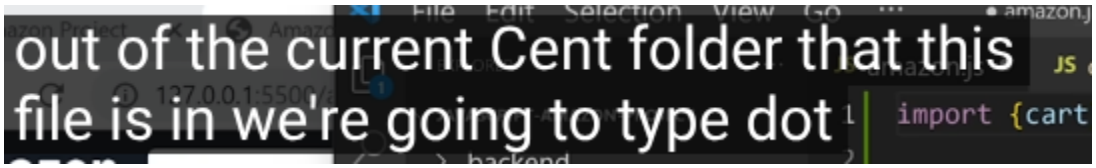


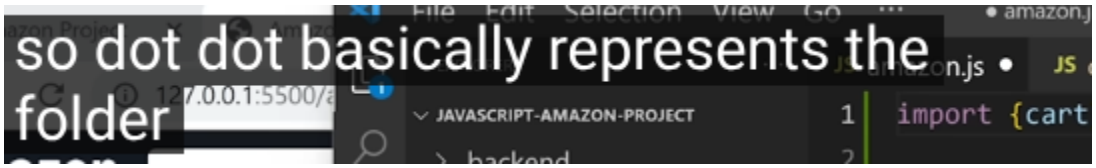
```
export const cart = [];
```

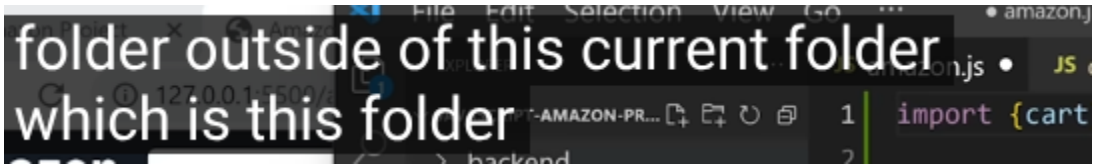
```
import{ // name of the var we want

}
```

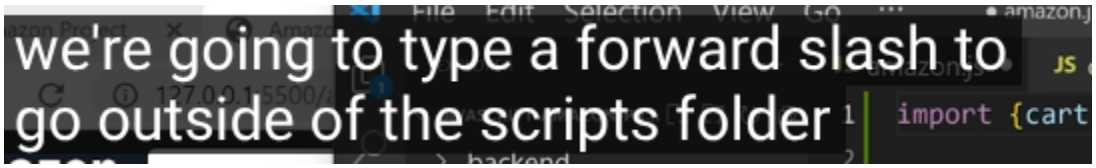
```
import{cart} from '../data/cart.js';
```

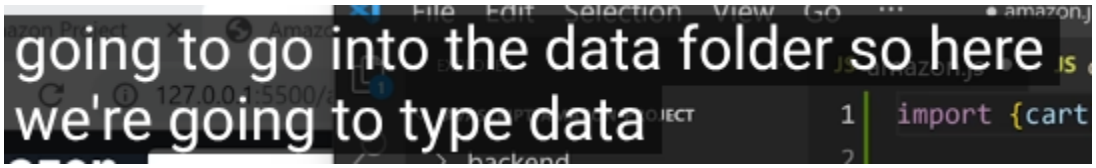










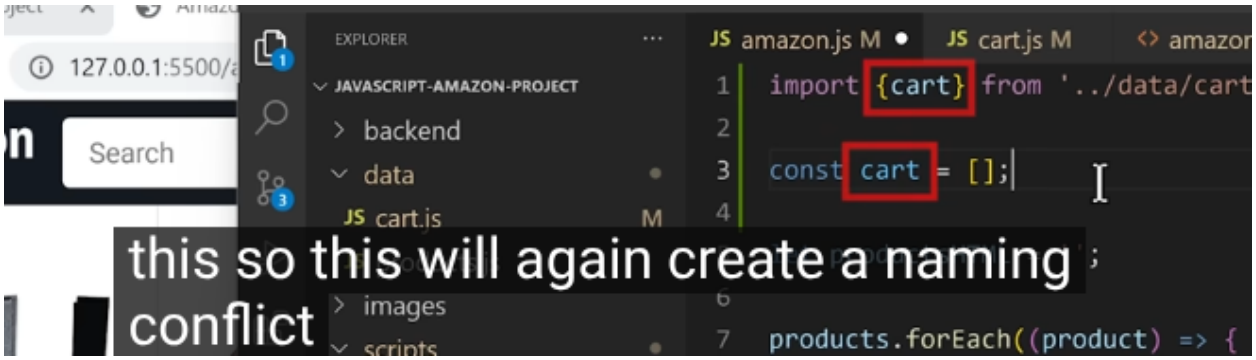


1. Put all imports at the top of the file.

2. We need to use Live Server

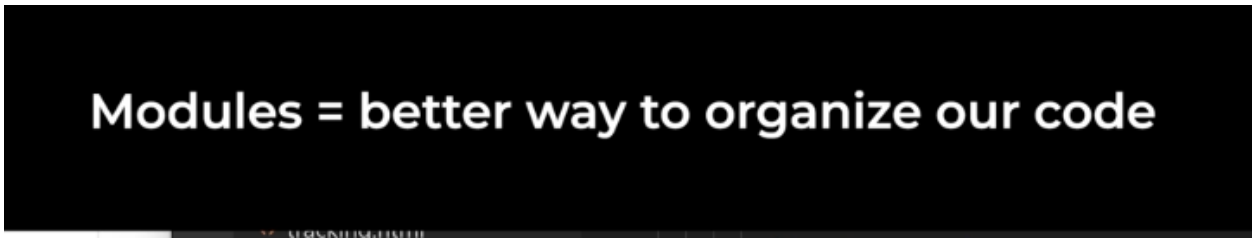
Benefits of Modules

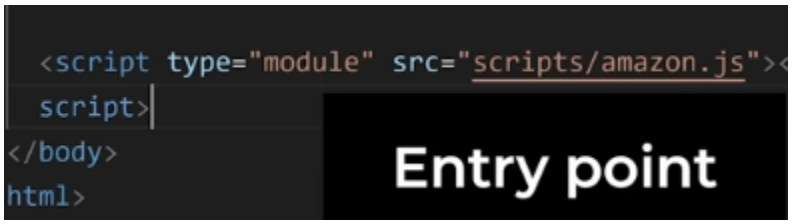
1. Helps us avoid naming conflicts

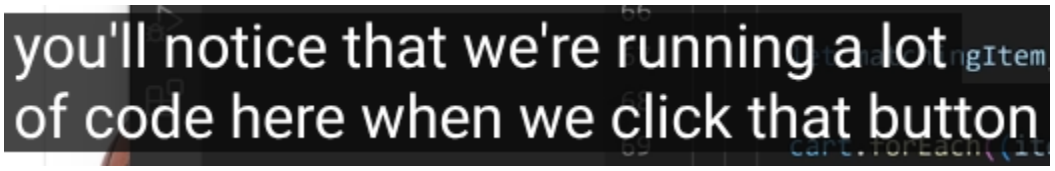


2. Don't have to worry about order of files

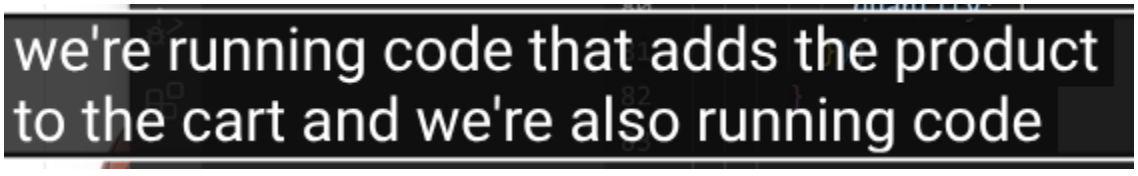

ipt tags we had to make sure that we
d C.J first because we need the cart
Have to load cart.js first
JS cart.js
1 const cart = [];
<script src="data/cart.js"></script>
<script src="data/products.js"></script>
<script src="scripts/amazon.js"></script>

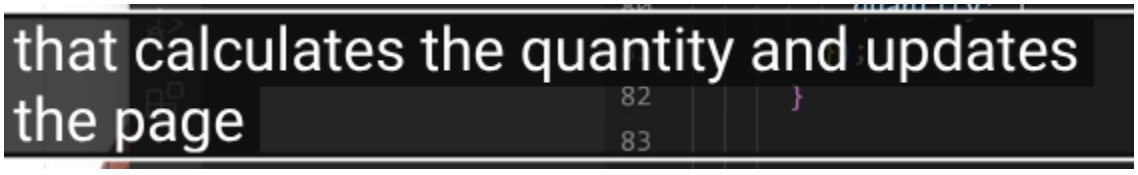

Modules = better way to organize our code


<script type="module" src="scripts/amazon.js"></
script>
</body>
html>
Entry point


you'll notice that we're running a lot
of code here when we click that button


we're running code that adds the product
to the cart and we're also running code


we're running code that adds the product
to the cart and we're also running code


that calculates the quantity and updates
the page

the page so a best practice in programming is that

programming is that when we have a lot of code here that does different

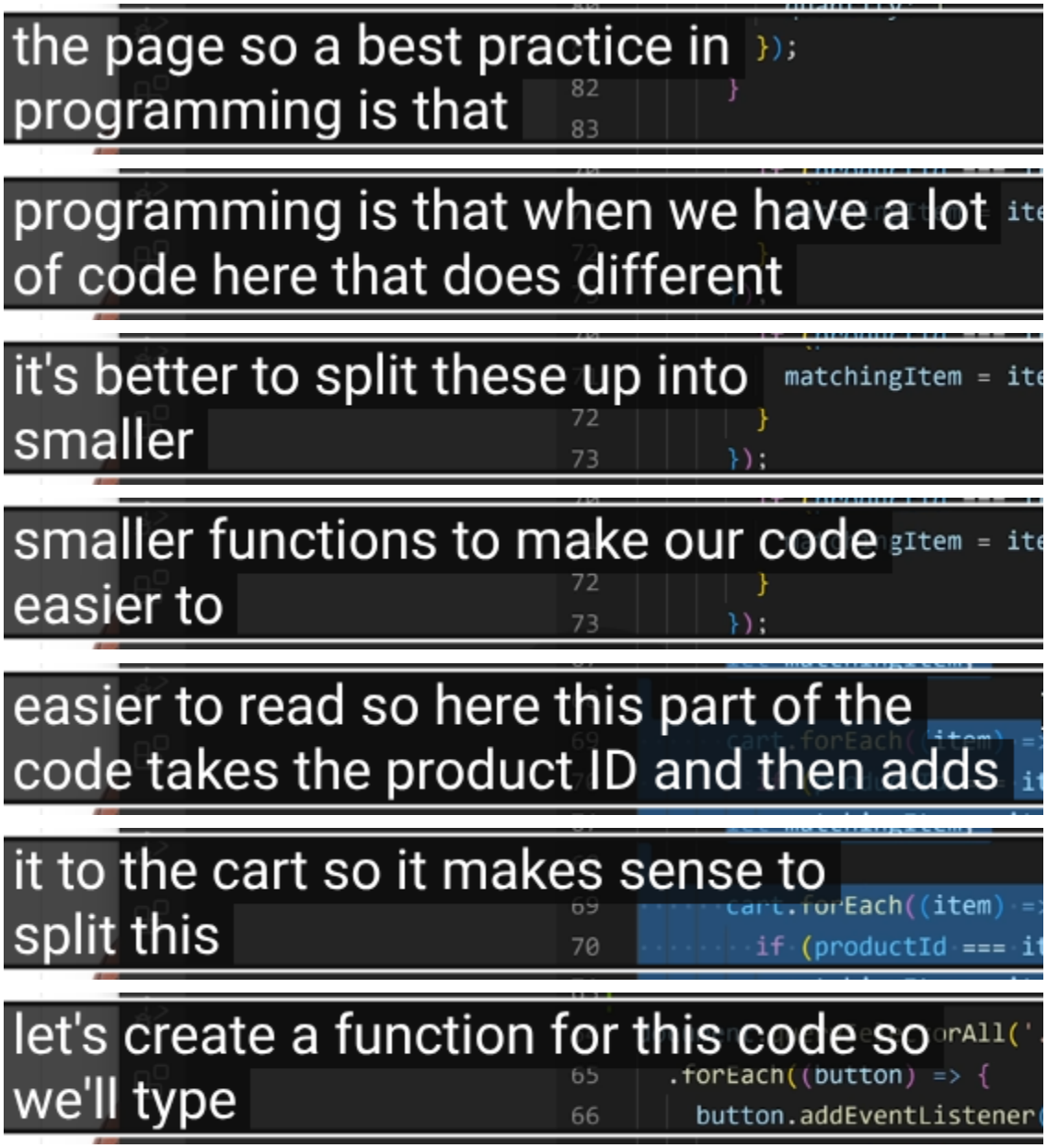
it's better to split these up into smaller

smaller functions to make our code easier to

easier to read so here this part of the code takes the product ID and then adds

it to the cart so it makes sense to split this

let's create a function for this code so we'll type



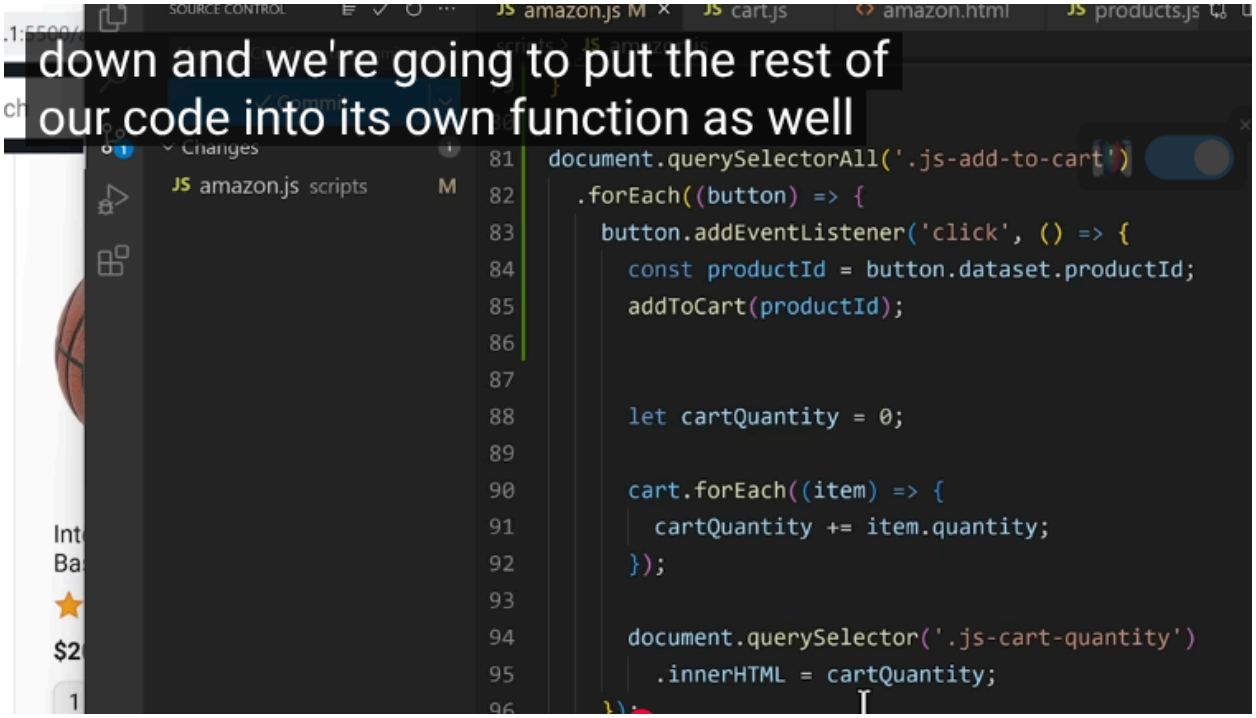
```
//---------- create another function for
readability--- //
function addToCart(){

    let matchingItem;

    cart.forEach((item)=>{
     if(productId === item.productId){
      matchingItem = item;
     }
    })

    if(matchingItem){
     matchingItem.quantity +=1;
    }
    else{
     cart.push ({
      productId: productId, // use productId
      instead of productName
      quantity : 1
     });
    }
};
```

```
document.querySelectorAll('.js-add-to-cart')
  .forEach((button)=>{
    button.addEventListener('click',()=>{
      const productId = button.dataset.productId;

      addToCart(); // call function // but not
      working

    let cartQuantity = 0;
    cart.forEach((item)=>{
      cartQuantity += item.quantity;
    });

    document.querySelector('.js-cart-quantity')
      .innerHTML = `${cartQuantity}`;
      console.log(cart);
  });
});
```

```
Unchecked runtime.lastError: The message port closed before a r
2 ▶ Uncaught ReferenceError: productId is not defined
      at addToCart (amazon.js:124:20)
      at HTMLButtonElement.<anonymous> (amazon.js:137:7)
```

```
  addToCart(productId); // call function //
  now working
```

```
 readability--- //
 function addToCart(productId){
```




```
function updateCartQuantity(){
    let cartQuantity = 0;
    cart.forEach((item)=>{
      cartQuantity += item.quantity;
    });

    document.querySelector('.js-cart-quantity')
      .innerHTML = `${cartQuantity}`;
      console.log(cart);
}

document.querySelectorAll('.js-add-to-cart')
  .forEach((button)=>{
    button.addEventListener('click',()=>{
      const productId = button.dataset.productId;
      addToCart(productId);
      updateCartQuantity();
  });
});
```

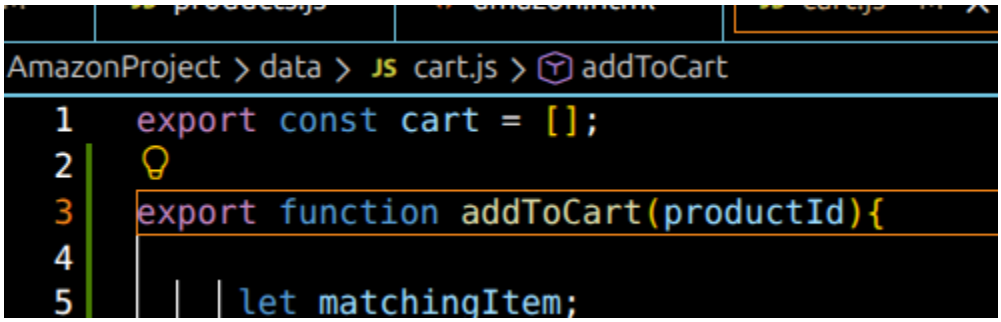
scroll up to the add to cart function so you'll notice that this code

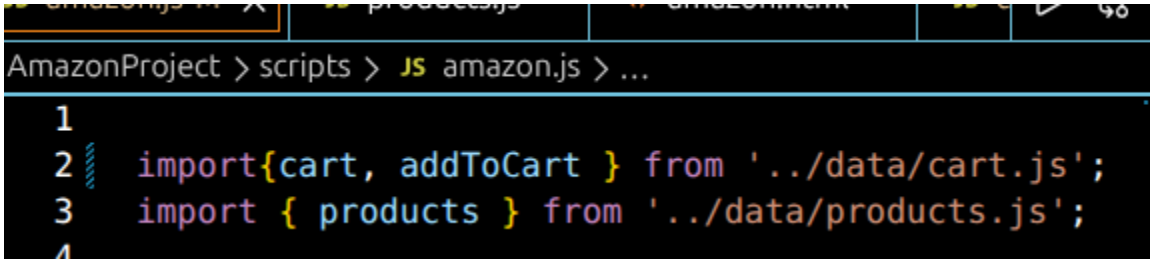
manages our cart so it's actually a best practice to move this function into

cart. JS because cart. JS contains all the code that's related to the cart

Best practice

Group related code together into its own file.





```
import * as cartModule from '../data/cart.js';

cartModule.cart
cartModule.addToCart('id');
```

file Imports have another syntax import Star as this Imports everything from a

file and groups it together inside this object and

```
66      cartQuantity += cartItem.quantity;
```

cart. JS as well so this function actually handles

```
66      cartQuantity += cartItem.quantity;
```

updating the web page rather than managing the cart so we're going to keep

```
66      cartQuantity += cartItem.quantity;
68
      document.querySelector('.js-cart-quantity')
```

this function inside this file for now so